

2025 Python Programming Language - Midterm Exam

(2025/11/10 13:10 - 16:10)

(A) Multiple Choice Questions (36%)

1. Which statement about Python **virtual environments** is most accurate? (A) Virtual environments affect the system-wide Python installation (B) Virtual environments are primarily used to isolate package dependencies between projects to avoid version conflicts (C) Only one virtual environment can be created per system (D) Virtual environments can only be used on Linux systems
2. Which statement about Python **indentation** is **incorrect**? (A) Python uses indentation to define code blocks (B) PEP 8 recommends using 4 spaces as the indentation standard (C) Tabs and spaces can be mixed for indentation in the same Python file (D) Indentation errors will cause an IndentationError
3. In which situation is a **while** loop more appropriate than a **for** loop? (A) Iterating through all elements in a list (B) Executing a loop for a fixed number of times (C) Uncertain number of iterations, stopping only when a specific condition is met (D) Iterating through multiple sequences simultaneously
4. Which statement about the advantages of **list comprehensions** is **least correct**? (A) Code is more concise and readable (B) Execution performance is usually better than traditional for loops (C) Can completely replace all for loop use cases (D) Suitable for simple data transformation operations
5. What is the primary difference between **tuples** and **lists** in Python? (A) Tuples can only store numbers, lists can store any type (B) Tuples are immutable, lists are mutable (C) Tuples use more memory than lists (D) Tuples do not support index access
6. Which statement about Python **f-string** formatting is correct? (A) f-strings were introduced in Python 2 (B) f-strings have better performance than % formatting and .format() method (C) f-strings do not support expression evaluation (D) f-strings can only be used for simple variable insertion
7. Which statement about function **parameter passing** is most correct? (A) Python uses pass by value (B) Python uses pass by reference (C) Python uses pass by object reference (D) The passing method depends on the parameter type
8. Which statement about Python **decorators** is **incorrect**? (A) Decorators are a design pattern for extending functionality without modifying the function itself (B) Decorators are essentially higher-order functions that receive and return functions (C) Only one decorator can be applied to a function (D) The @ symbol simplifies decorator application
9. Which situation is most suitable for using **lambda expressions**? (A) Complex functions that need to be reused in multiple places (B) As simple parameters for higher-order functions (like map, filter) (C) Functions containing multiple conditional branches (D)

Functions requiring detailed documentation

10. What is the main advantage of using the **with statement** for file handling? (A) Can open multiple files simultaneously (B) Automatically handles file closing, ensuring resource release even if exceptions occur (C) Improves file read/write execution speed (D) Can handle all types of file formats
11. Which statement about **class methods** and **static methods** in Python is correct? (A) Class methods use `@staticmethod` decorator, static methods use `@classmethod` decorator (B) Class methods must have `cls` as first parameter, static methods don't need a special first parameter (C) Class methods and static methods have identical functionality (D) Only static methods can access class attributes
12. Which statement about **polymorphism** in Python is most correct? (A) Python does not support polymorphism (B) Polymorphism can only be achieved through inheritance (C) Python uses duck typing, allowing polymorphism without explicit inheritance (D) Polymorphism significantly reduces program execution efficiency

(B) Code Completion (30%)

1. There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

- Input: `ratings = [1, 0, 2]`
- Output: 5
- Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

- Input: `ratings = [1, 2, 2]`
- Output: 4
- Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively. The third child gets 1 candy because it satisfies the above two conditions.


```

class Solution:
    def candy(self, ratings: List[int]) -> int:
        n = len(ratings)
        total_candies = n
        i = 1

        while i < n:
            if ratings[i] == ratings[i - 1]:
                i += 1
                continue

            current_peak = 0
            while i < n and ratings[i] > ratings[i - 1]:
                current_peak += 1
                total_candies += current_peak
                i += 1

            if ratings[i] < ratings[i - 1]:
                current_valley = 0
                while i < n and ratings[i] < ratings[i - 1]:
                    current_valley += 1
                    total_candies += current_valley
                    i += 1

            total_candies += current_peak + current_valley

        return total_candies

```

2. You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return true if you can reach the last index, or false otherwise.

*Hint (If we are at a position `k` and `nums[k] = 4`, then it means we can jump forward a maximum of 4 steps from this position. It's our choice to jump 1,2,3 or 4 positions, but not more than 4.)

Example 1:

- Input: `nums = [2, 3, 1, 1, 4]`
- Output: true

Example 2:

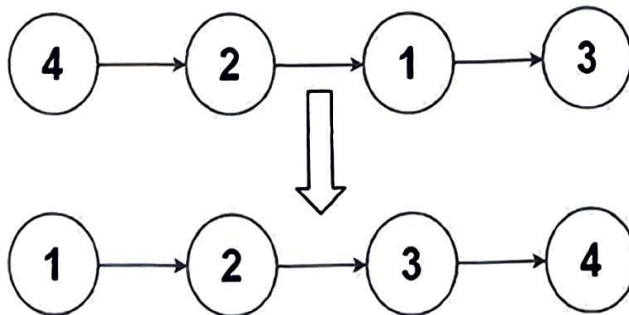
- Input: nums = [3, 2, 1, 0, 4]
- Output: false

class Solution:

```
def canJump(self, nums: List[int]) -> bool:
    n = len(nums)
    maxreach = 0
    last = _____(5)
    for i in range(n):
        if _____(6):
            return False
        maxreach = max(_____(7), i + nums[i])
        if maxreach >= last:
            return True
    return False
```

3. Given the head of a linked list, return the list after sorting it in ascending order.

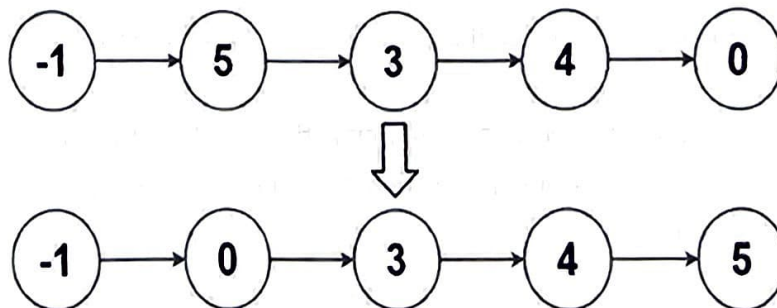
Example 1:



Input: head = [4,2,1,3]

Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]

Output: [-1,0,3,4,5]

```

class Solution:
    def sortlist(self, head: ListNode) -> ListNode:
        if not head or not head.next:
            return head
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        mid = slow.next
        slow.next = None
        left = sortlist(head)(8)
        right = sortlist(mid)(9)
        return merge(left, right)(10)

    def merge(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        tail = dummy

        while l1 and l2:
            if l1.val < l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next

        tail.next = l1 if l1 else l2
        return dummy.next

```

(C) Programming Exercise (40%)

1. (15%) Report Writing

You are a student facing the final submission deadline for a critical term paper. Your goal is to compile your research materials into the complete **Target Report Content T**. Time is your score — you must determine the **absolute minimum time** required to finalize your report.

While constructing the report, you can input content using two different methods, each with its own fixed time cost:

➤ **Keyboard Typing Input**

- You type diligently, and every single character (letter, number, or punctuation mark) takes **0.3 seconds** to enter.

➤ **Key Term Shortcut Paste**

- Your note-taking software has a pre-defined “**Key Term**” substring C that you’ve prepared for this report.

- You can use a dedicated shortcut to paste it, and each paste action regardless of the length of C — takes **0.4 seconds**.
- This shortcut can be used an unlimited number of times.

Your task is to determine the **fastest possible** way to construct the target content T, and compute the **minimum total time required**.

Input Format

The input consists of two lines:

The first line is the Target Report Content T. It includes English letters, numbers, punctuation marks, spaces, etc., and should be read as a whole line.

The second line is the Key Term Shortcut C. It includes English letters, numbers, punctuation marks, spaces, etc.

Output Format

A minimum time in seconds, formatted as a floating-point number and rounded to one decimal place

Example:

➤ Input:

The sunsets in the west. The river flows to the sea The answer is found in the data.
the

➤ Output:

24.0

2. (15%) Spiral Matrix Generator

You are conducting a small visualization experiment with grid data. To observe the pattern of number distribution, you decide to fill a square matrix in a special order. The process starts from the center of the matrix, placing the number 1 in the center, and then the numbers expand outward in a **counterclockwise spiral** pattern, forming an elegant, organized structure until the entire matrix is filled.

Center Position Definition:

- If n is odd: The center is the exact middle cell of the matrix, specifically at position $(n/2, n/2)$ (using 0-based indexing).
- If n is even: The center is defined as the lower-left cell of the central four cells, specifically at position $(n/2, n/2-1)$ (using 0-based indexing).

Rules: Start by placing the number 1 at the center, then fill in 2, 3, 4, ..., n^2 following a fixed counterclockwise direction pattern.

- Direction Sequence: Right → Up → Left → Down (This sequence repeats continuously).

Input Format

A single line containing one integer: n ($1 \leq n \leq 99$)

Output Format (Alignment Required)

Output an $n * n$ matrix, consisting of n rows, with n integers per row.

Formatting Requirements

1. Right-Alignment: All numbers must be **properly right-aligned**.
2. Column Width: Let $w = \text{len}(\text{str}(n * n))$ be the column width (i.e., the number of digits in the largest number). Each number should take exactly w spaces.
3. Separation: Numbers are separated by a **single space**.
4. No Trailing Spaces: No extra spaces are allowed at the end of each line.

Example 1:

- Input: 3

- Output:

5 4 3

6 1 2

7 8 9

Example 2:

- Input: 4

- Output:

16 15 14 13

5 4 3 12

6 1 2 11

7 8 9 10

3. (10%) Haunted Manor Path Counter

You and your classmates, aiming for an ultimate Halloween thrill, decide to take on the most notorious haunted building in town: the **Blackwood Manor**. The manor's owner is said to be cursed by an evil spirit, turning the entire mansion into a maze full of traps and unknown horrors.

Your Mission: Starting from the entrance (top-left corner), you must find a safe way to the emergency exit (bottom-right corner).

Center Position Definition:

- The entire manor is represented by an $N \times N$ grid.
- 0 (Open Path): Safe hallways or rooms where you can pass through easily.
- 1 (Sealed Wall): These are walls sealed by the spirit's power, collapsed areas, or hidden traps that you cannot cross.
- You are only allowed to move in four primary directions: **Up, Down, Left, and Right.**

Input Format

The first line contains a single integer: n

The next n lines each contain n integers (0 or 1), separated by spaces, representing the layout of the manor.

Output Format

Output a single integer — the number of different possible paths from the entrance (top-left) to the exit (bottom-right).

If no path exists, output 0.

Example:

➤ Input:

```
5
0 0 1 0 0
0 0 1 0 1
1 0 1 1 0
1 1 0 0 0
0 0 0 1 0
```

➤ Output:

```
2
```